

Python Exception Handling: Complete Concepts with Examples

1. What is an Exception?

- An exception is an error that occurs during the execution of a program.
- Python uses exceptions to handle runtime errors.

2. Basic Syntax

```
try:  
    # Code that may raise an exception  
except SomeException:  
    # Code to handle the exception  
finally:  
    # Code that always executes
```

3. Try Block

- Contains the code which may raise an exception.
- If no exception occurs, the except block is skipped.

Example:

```
try:  
    x = int(input("Enter a number: "))  
    print(10 / x)  
except ZeroDivisionError:  
    print("Division by zero not allowed")  
finally:  
    print("Execution completed")
```

4. Except Block

- Handles the exception.
- Can specify the type of exception to catch.

Example:

```
try:  
    num = int(input())  
except ValueError:  
    print("Please enter a valid number")
```

5. Catching Multiple Exceptions

```
try:
    x = int(input())
    print(10 / x)
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Division by zero")
```

6. Generic Exception

- Catch all exceptions using `Exception`.
- Useful for custom exceptions or unknown errors.

```
try:
    raise Exception("Something went wrong")
except Exception as e:
    print(e)
```

7. Exception Object and `as e`

- When an exception is raised, it is an object of a class.
- `as e` stores the exception object for accessing message.

```
try:
    raise Exception("Custom error")
except Exception as e:
    print("Caught:", e)
```

8. Finally Block

- Executes whether exception occurs or not.

```
try:
    x = 10 / 2
except ZeroDivisionError:
    print("Error")
finally:
    print("Always executed")
```

9. Raising Custom Exceptions

- Use `raise` to throw exceptions.

```
balance = 1000
credit = 2000
if credit > balance:
    raise Exception("Insufficient balance")
```

10. Custom Exception Class

```
class ATMEError(Exception):
    pass

try:
    raise ATMEError("Not enough balance")
except ATMEError as e:
    print(e)
```

11. Key Points

1. Exceptions are objects with type and message.
2. `except` block selection is **type-based**, not message-based.
3. `as e` allows access to exception object.
4. `finally` always executes.
5. Can catch multiple exceptions with multiple `except` blocks.
6. Generic `Exception` catches all types of errors.

12. Real-Life Example (ATM System)

```
try:
    balance = 10000
    credit = int(input("Enter credit amount: "))

    if credit > balance:
        raise Exception("Insufficient balance")
    if credit <= 100:
        raise Exception("Amount too small")

except ValueError as e:
    print("Invalid input, enter a number")
except Exception as e:
    print(e)
finally:
    print("Thank you for visiting the ATM")
```